CAMPUS FEEDBACK LENS

PROJECT REPORT

HTML · CSS · JavaScript · PHP · MySQL

Abstract

Campus Feedback Lens is a web-based Student Feedback Analysis System designed for colleges and universities. It provides a structured process for collecting student opinions about teaching quality, communication, punctuality, subject knowledge and classroom interaction. Students submit feedback through a role-based portal. Faculty members review aggregated results and anonymous comments, while administrators manage users, subjects and institutional reports. The application is implemented using HTML, CSS, JavaScript, PHP and MySQL. It includes password hashing, prepared database statements, role-based access checks, CSRF validation and database-level duplicate submission prevention.

1. Introduction

Student feedback is an important input for improving teaching and learning. When feedback is collected manually, institutions may face delays, inconsistent formats, difficulty comparing departments and concerns about student privacy. Campus Feedback Lens provides a central digital platform to collect consistent feedback and convert it into useful summaries for academic review.

2. Problem Statement

Colleges need a convenient and consistent method to collect student feedback and analyse the results. Manual forms and spreadsheets can be difficult to manage, can expose individual responses and make it hard to compare performance across subjects or sessions. The proposed system addresses these challenges through a role-based web application and aggregated reporting.

3. Objectives

• Provide an easy-to-use online feedback form.

• Rate five teaching-related criteria on a 1–5 scale.

• Prevent duplicate feedback for the same student, subject and session.

• Protect student identity in faculty and management views.

• Provide faculty and department-level summaries.

• Allow administrators to manage users and subjects.

• Export aggregate reports to CSV and PDF through browser printing.

4. Scope

The prototype includes student, faculty and administrator modules. Students can view active subjects, submit feedback and see their own submission history. Faculty can review aggregate ratings and anonymous comments. Administrators can review campus metrics, manage accounts and subjects, and export reports. Integration with a college ERP, email notifications, advanced sentiment analysis and institution-wide single sign-on are outside the current scope.

5. Technology Stack

Frontend: HTML5, CSS3 and vanilla JavaScript. Backend: PHP 8.1 or later. Database: MySQL 8 or compatible MariaDB. Server: Apache through XAMPP/WAMP or an equivalent PHP web server. Database access uses PHP PDO and prepared statements.

6. System Architecture

The browser renders PHP-generated HTML and uses CSS and JavaScript for responsive presentation and client-side interactions. PHP page controllers validate user input, verify session roles and CSRF tokens, and communicate with MySQL through PDO. The database stores departments, user accounts, subjects and feedback. Faculty and admin reports query aggregate ratings without selecting student identity fields.

7. Modules

Student module: authentication, subject listing, feedback submission and personal submission history. Faculty module: response totals, average ratings by subject and anonymous comments. Admin module: campus overview, department comparisons, account creation and subject creation. Reporting module: department/faculty/session filters, aggregate CSV download and browser print-to-PDF.

8. Database Design

Department stores academic units. User stores name, unique username, password hash, role and optional department. Subject links a subject code and name to a faculty member and department. Feedback stores the subject, faculty, academic session, five rating values, optional comment and creation timestamp. A unique constraint on student_id, subject_id and session_name prevents duplicate submissions. The student_id field is internal and is not included in faculty or management report queries.

9. Privacy and Security

Passwords are stored using PHP password_hash and checked with password_verify. Database operations use prepared statements. Protected pages enforce role checks. POST forms validate CSRF tokens. Output is HTML-escaped. The feedback table retains an internal student reference for duplicate prevention; therefore the implementation provides anonymity in faculty-facing reports, not absolute anonymity from privileged database administrators. Production deployment should use HTTPS, secure cookies, rate limiting, backups, retention policies and least-privilege database credentials.

10. Installation and Execution

Install XAMPP and start Apache and MySQL. Create a MySQL database named campus_feedback_lens. Import database/schema.sql. Configure DB_HOST, DB_NAME, DB_USER and DB_PASS in config/config.php. Copy the project into the web server document root and run setup.php once to create demo accounts and sample departments/subjects. Delete setup.php after installation. Open the application in a browser and sign in using the demo credentials listed in README.md.

11. Testing

The test plan covers successful and failed login, role restrictions, feedback submission, rating validation, duplicate prevention, anonymity of faculty views, user and subject creation, CSV export and print-to-PDF. Refer to docs/Test-Cases.md for the complete test-case table.

12. Limitations and Future Enhancements

The current prototype uses a simple academic session text field and demo accounts. Future improvements may include email verification, password reset, bulk user import, subject enrolment rules, response-window scheduling, chart-based trend visualisations, sentiment analysis, multilingual forms, notification services, audit logs and integration with institutional systems.

13. Conclusion

Campus Feedback Lens demonstrates how a web-based feedback platform can simplify feedback collection and make academic insights easier to review. The role-based design, aggregate reporting and duplicate submission

constraint support a consistent feedback workflow. With additional operational security and institutional integration, the prototype can be extended into a production-ready campus feedback service.

Appendix A – Demo Accounts

Role	Username	Password
Student	student	student123
Faculty	faculty	faculty123
Admin	admin	admin123